MCMC Metropolis-Hastings

Alejandro Romero Mora

15 de octubre de 2026

1. General theory

As the purpose of this document is to explain a Python program and the algorithm on which it is based, some concepts will be assumed to be known to the reader.

1.1. Bayesian parameter inference

Bayes Theorem — $P(A|B)P(B) = P(B|A)P(A)$ — becomes interesting when A is replaced by a set of parameters θ and B by the observed data or the samples, denoted as d. Then, Bayes Theorem can be written as

$$P(\theta|d) = \frac{P(d|\theta)P(\theta)}{P(d)}. \tag{1}$$

$P(\theta|d)$ is the posterior probability for θ, and it represents the degree of belief about the value of θ after seeing the observed data. $P(d|\theta) = L(\theta)$ is the likelihood function, i.e. the probability of the data given a certain value for the parameters.
$P(\theta)$ is called the prior, and it represents the degree of belief in the value of θ before seeing the data. When running an algorithm, the shape of the initial prior has to be chosen by the user, but the posterior generally converges to a prior-independent function for large datasets. The probability $P(d)$ is a normalizing constant, often called evidence.

Then, Bayes Theorem relates the information about the parameters after seeing the data to the likelihood and the prior, known before observing the data, updating the state of knowledge about a parameter as new data is introduced.

1.2. MCMC methods

A Markov Chain is a sequence of random variables $\{X^{(0)}, X^{(1)}, ..., X^{(N)}\}$ such that the probability of the $(t+1)$-th element in the chain only depends on the value of the t-th element. They can converge to a stationary state, i.e. the chain does not change with t, and the successive elements of the chain are samples from the posterior.

The MCMC (Markov Chain Monte Carlo) methods are a family of statistical methods where the next step is generated from the current one, building a chain in parameter space, whose crucial property is that the density of samples (the frequency

in which the samples fall within certain regions of the parameter space) is proportional to the posterior. This allows a fast convergence in high-dimensionality problems, where other algorithms might require too much computational power.

In the Metropolis-Hastings method — one of the most used MCMC algorithms — when a new step is performed, the candidate point is accepted with probability

$$\alpha = min\left(\frac{p_c}{p_i}, 1\right), \tag{2}$$

where $p_c = p(\theta^{(c)}|d)$ is the posterior probability at the candidate point, and p_i is the probability of current step. If the candidate point performs better than the current point, it is accepted and added to the chain, and if $p_c < p_i$, then there is a probability of p_c/p_i for the point to be accepted. This allows the algorithm to escape from a local maximum and explore all the space of parameters.

The principal advantage of MCMC methods is its great performance in high-dimensional problems: in traditional methods, the number of steps the program takes scale rapidly with the number of dimensions. MCMC does not try to explore all the parameter space; it tries to converge to high-probability regions.

In parameter spaces with complex topologies, MCMC has the capacity to skip from a local maximum to the global maximum thanks to the Metropolis Hastings algorithm.

This process can be adapted to solve problems with higher dimensionality, where other algorithms are not capable to converge so quickly. MCMC also captures very well the degeneracies, letting see how does a parameter depends on the other. Previous information about the knowledge of the parameters can be easily included in the sampling by the use of priors.

That is the reason why MCMC is widely used in industry for decision-making under uncertainty, including Bayesian marketing mix models and A/B testing, credit risk and actuarial modeling in finance and insurance. It is also applied in engineering and logistics for reliability analysis and demand forecasting.

2. About the program

The objective of the program is to make a Metropolis-Hastings MCMC algorithm to solve a linear model. This solution can be obtained analytically or using simpler methods, but as the model can be changed by defining a new function, the purpose of the program is to serve as a simple example of how a MCMC algorithm works.

The function `log_likelihood` takes a gaussian centered in each $\mu_i = \theta_0 + \theta_1\hat{x}_i + \epsilon_i$, where $\epsilon_i \sim N(0, \sigma)$ is the gaussian noise. Then, the function calculates the deviation from the expected value y via maximum log-likelihood (MLL). If the data point is close to the prediction, the value of the log-likelihood is high and vice versa.

The function `log_prior` sets the allowed limits for each parameter using a uniform distribution. Then, `log_posterior` calculates the log-posterior summing the log-prior

and the log-likelihood.

The function `metropolis` contains the Metropolis-Hastings algorithm. The initial parameters to start iterating are obtained numerically using the function `minimize` directly from the Likelihood. In this case we can do that because the maximum is unique. Then, these parameters, with a little gaussian noise (to force the program to explore the parameter space), are introduced in the MCMC function, obtaining an array with all the new parameters obtained in the iterations of the function. Then, I calculated the mean of the parameters' array, removing the first 20 % of the iterations (burn-in), starting from where the chain should have reached the stationary distribution of the posterior so it should have converged. It is also made to eliminate the dependence from the initial conditions.

3. How to use the program

All the changes related with the input parameters can be changed in the section #INPUTS#, between lines 10 and 46.

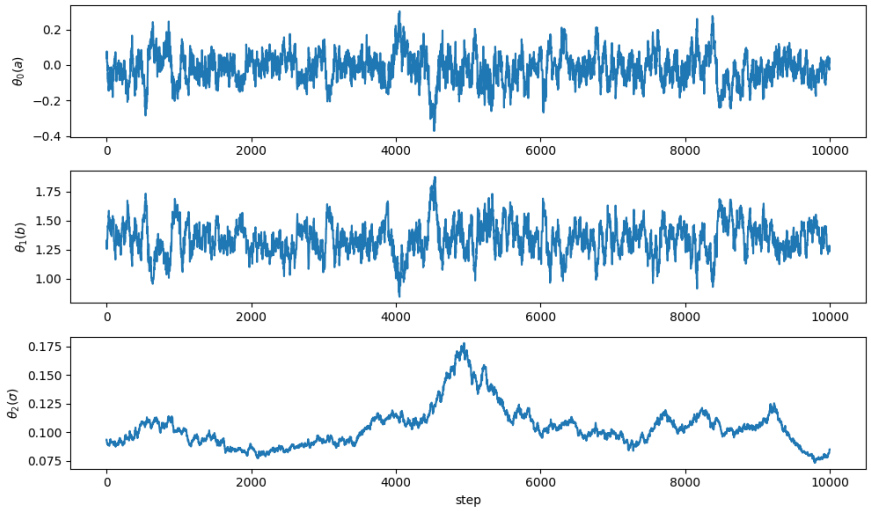
The line 37, `np.random.seed(42)`, is responsible for ensuring that the program produces the result shown in this document. If a random result is needed to evaluate the performance of the program, this line has to be removed.

The number of steps taken by the algorithm, the limits allowed for each parameter, the size of the steps or the burn in can also be changed in this section.

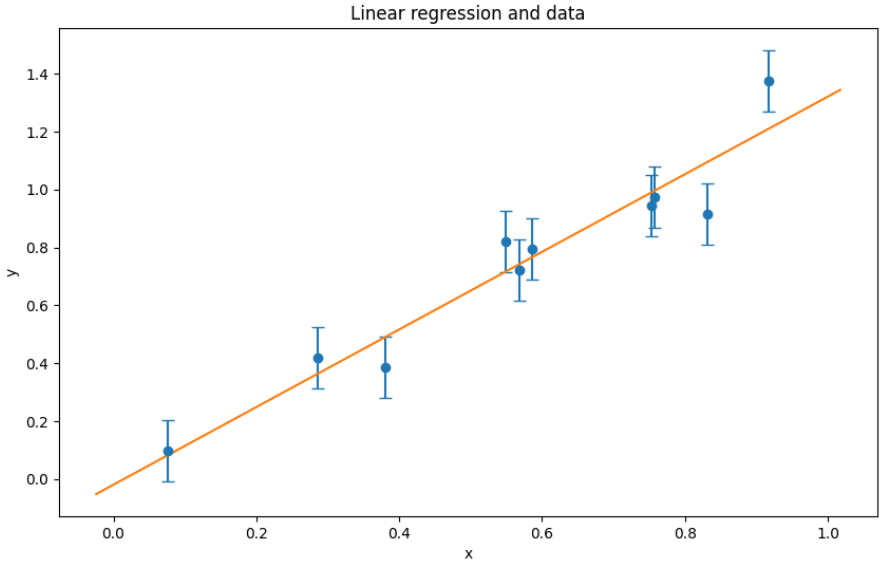
If a different model is wanted (quadratic, exponential, etc.), the new model has to be defined, and the parameters have to be named as `theta[i]` with i from 0 to the required value. The different inputs for x and y (or the variables required) have to be changed to adapt the inputs with the model if an accurate result is expected. Then, in lines 106 and 120, `linear_model` has to be substituted by the new function. The different bounds and step sizes should also be adapted to the new data entered. For simplicity, the parameters related to the plot of the covariance ellipse, so if necessary, it has to be changed specifying the parameters wanted to be plotted.

4. Results

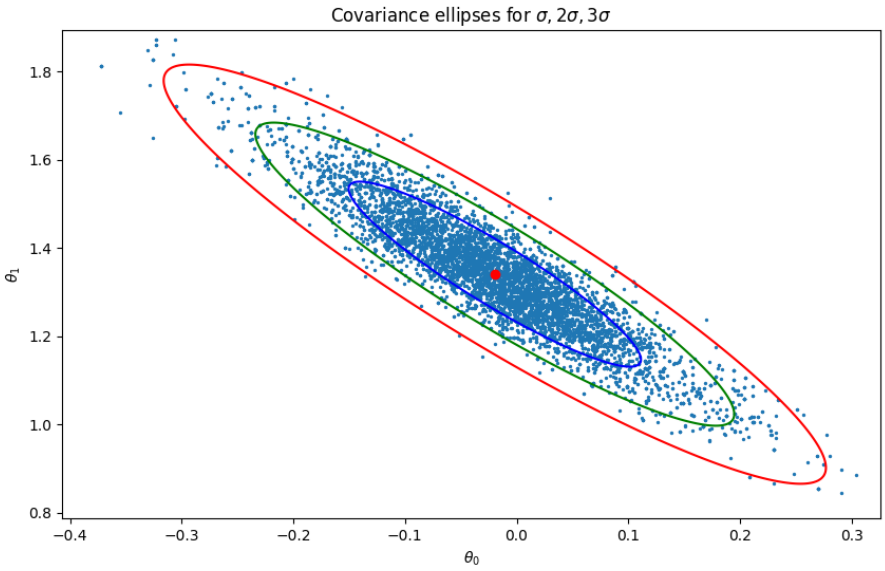
In the next figure is shown the evolution of the three parameters through the steps of the chain.



Then, the data and the linear regression using the uncertainty obtained by the MCMC:



Finally, the covariance ellipses with the points generated by the algorithm:



In the next table the principal results estimated by the program, compared with the ones made by a linear regression are shown:

Parameter	**Mean (Uncertainty 68 %)**	**Analytical results**
θ_0	-0.0195 (+0.0301 / -0.0837)	-0.013
θ_1	1.3412 (+0.0441 / -0.1290)	1.331
σ	0.1059 (+0.0027 / -0.0173)	0.093

The obtained values are consistent with the expected ones within the 1σ (68 %) credibility interval.